

Absolutely, buddy. 🤖 **I'll take the lead from here.**

We will treat this as an engineering checkpoint, not an open-ended coding exercise. I'm going to define CP-MVP2-03 first, freeze its contract, and then give Claude a tightly bounded implementation instruction.

📄 CP-MVP2-03 — Contract & Implementation Plan

1. Objective

CP-MVP2-03 = LLM Testcase Generation

Its sole responsibility is:

Generate governed, traceable testcases from requirements retrieved through the frozen MVP2 Knowledge Base/RAG layer.

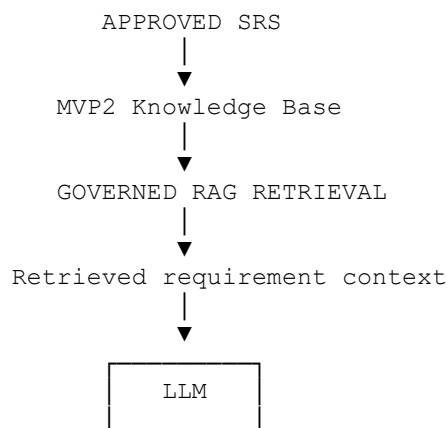
It is **not** responsible for:

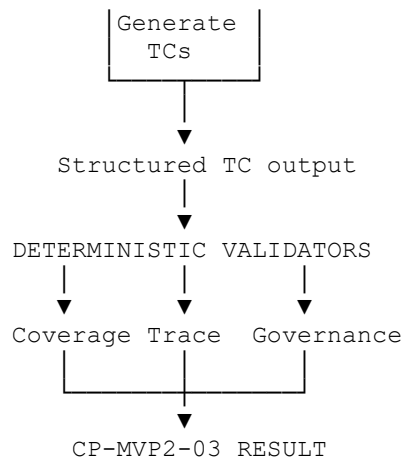
- browser automation
- API execution
- test-data execution
- performance testing
- security execution
- self-healing
- Git PR creation
- modifying requirements
- discovering new Demo Web Shop functionality

Those belong to later CPs.

2. Authoritative flow

This is the architecture I want us to implement:





The key architectural boundary is:

RAG supplies the evidence. LLM performs the reasoning. Deterministic code decides whether the output complies.

3. Input contract

CP-MVP2-03 should consume only controlled inputs.

Required

A. Requirement source

From the frozen MVP2 knowledge base/RAG.

Each retrieved requirement should carry enough attribution to establish:

```
requirement_id
requirement_text
source_document
source_version
approval_status
retrieval metadata
```

B. Generation request

Something conceptually like:

```
Generate testcases for:
<requirement scope>
```

Potentially with controlled parameters such as:

```
scenario_types:
- positive
```

- alternate
- exceptional

But these parameters must **not alter the requirements**.

4. Testcase schema

I recommend we establish a structured JSON contract rather than allowing the LLM to return free-form prose.

A testcase should contain at least:

```
testcase_id
title
requirement_ids
journey_id
scenario_type
preconditions
test_steps
expected_result
test_data_reference
priority
source_attribution
generation_metadata
```

For example:

```
{
  "testcase_id": "TC-001",
  "title": "...",
  "requirement_ids": ["REQ-001"],
  "journey_id": "JRN-001",
  "scenario_type": "positive",
  "preconditions": [],
  "test_steps": [],
  "expected_result": "...",
  "test_data_reference": [],
  "priority": "High",
  "source_attribution": [],
  "generation_metadata": {}
}
```

Important: we don't need to hard-code this exact JSON yet. Claude should first inspect the existing CP-MVP2-01/02 contracts and reuse their conventions.

That follows our **"don't reinvent the wheel"** principle.

5. Three classes of generated scenarios

The LLM should reason about three categories.

Positive

The primary successful business flow explicitly supported by the requirement.

Alternate

A valid variation of the requirement.

Exceptional

An error/invalid/boundary condition **only when supported or reasonably derived from the requirement's stated behavior/constraints.**

This distinction is important.

We don't want:

"LLM knows software testing, therefore generate every conceivable negative testcase."

We want:

"Given this requirement and its governed context, what tests are justified?"

6. Traceability is mandatory

Every testcase must be traceable backward.

```
TC
↓
REQ
↓
Approved SRS
↓
Source document/version
```

No orphan testcase should be accepted.

Therefore deterministic validation should reject something like:

```
TC-017
requirement_ids: []
```

Likewise, if:

```
requirement_ids: ["REQ-999"]
```

doesn't exist in the governed requirement set, it must fail validation.

7. Coverage model

This is where I want CP-MVP2-03 to become genuinely useful.

We should calculate:

```
Requirement coverage =  
requirements having ≥1 valid testcase  
-----  
total applicable requirements
```

And report:

```
Total requirements      : N  
Covered requirements    : X  
Uncovered requirements  : Y  
Coverage                : Z%
```

But 100% numerical coverage alone is not sufficient.

A testcase with a fake requirement ID must never count.

So:

```
VALID TRACEABILITY  
  +  
VALID TESTCASE  
  =  
COVERAGE
```

8. Unsupported requirements

Suppose the RAG retrieves:

REQ-015

but the generator cannot produce a valid testcase from the available requirement evidence.

We do not allow the LLM to invent one.

Instead:

```
REQ-015  
Status: UNSUPPORTED / NEEDS CLARIFICATION  
Reason: insufficient requirement information
```

This becomes valuable Agentic QE behavior.

The system is allowed to say:

"I don't have enough governed evidence."

That is far better than hallucinating a testcase.

9. Parked/draft leakage protection

This is particularly important because we already have Approved vs Draft governance.

CP-MVP2-03 must prove:

```
APPROVED
  ↓
eligible

DRAFT
  ↓
NOT eligible
```

If draft information appears in retrieved context accidentally, deterministic governance should prevent it from becoming an accepted testcase.

This should be an explicit test.

10. Duplicate testcase detection

We should have deterministic checks for obvious duplicates and preferably semantic similarity where practical.

At minimum:

```
same requirement
+
same scenario
+
same expected outcome
```

should not create multiple equivalent TCs merely because the LLM worded them differently.

The result should distinguish:

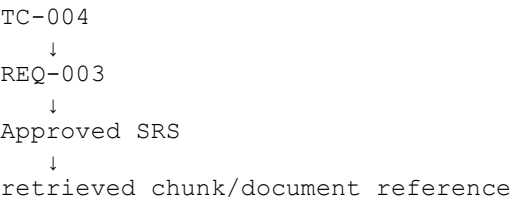
```
UNIQUE
DUPLICATE
POSSIBLE_DUPLICATE
```

I don't want an aggressive fuzzy algorithm causing legitimate scenarios to disappear.

11. Attribution

Every generated testcase should preserve evidence of **why it exists**.

Conceptually:



This is essential when a human reviewer asks:

"Why did the agent generate this testcase?"

We should be able to answer immediately.

12. LLM responsibilities vs deterministic responsibilities

This is the most important design table for CP-MVP2-03.

Responsibility	LLM	Deterministic code
Understand requirement	✓	
Reason about scenarios	✓	
Generate testcase	✓	
Generate steps	✓	
Generate expected outcome	✓	
Map to requirement	Assist	✓ Validate
Validate requirement exists		✓
Calculate coverage		✓
Detect duplicate IDs		✓
Validate schema		✓
Detect unsupported requirement Assist		✓

Responsibility	LLM Deterministic code
Prevent draft leakage	✓
Governance	✓
Produce audit result	✓

This gives us a very strong enterprise architecture.

13. CP-MVP2-03 acceptance gate

I propose we don't declare this checkpoint complete unless all of these pass:

Functional

- LLM generation works.
- Structured testcase output is produced.
- Positive scenarios work.
- Alternate scenarios work where applicable.
- Exceptional scenarios work where justified.

Governance

- Only approved governed requirements are consumed.
- Draft requirements cannot become accepted TCs.
- Unknown requirement IDs are rejected.
- Unsupported requirements are explicitly reported.
- No requirement invention.

Traceability

- Every accepted TC maps to a valid requirement.
- Source attribution exists.
- Requirement → TC coverage can be calculated.

Quality

- Duplicate detection works.
- Schema validation works.
- Coverage calculation works.
- Deterministic validation works.

Regression

Most importantly:

All existing CP-MVP2-01 and CP-MVP2-02 tests continue to pass.

We don't build CP-MVP2-03 by breaking the foundation.

14. Scope boundary

Claude needs this instruction very explicitly:

DO NOT START:

```
CP-MVP2-04
test execution
browser automation
API automation
test-data execution
performance execution
security execution
self-healing
CI/CD orchestration
PR automation
```

CP-MVP2-03 ends at:

```
REQUIREMENT
↓
RAG
↓
LLM
↓
TESTCASE
↓
VALIDATION
↓
GOVERNED OUTPUT
```

15. Claude implementation instruction

This is the instruction I would give Claude **after we point it at the GitHub repository**:

CP-MVP2-03 — LLM Testcase Generation

Objective

Implement CP-MVP2-03 as the first downstream Agentic QE capability on top of the frozen MVP2 Knowledge Base and governed RAG implementation.

The capability shall generate structured testcases from requirements retrieved through the existing governed RAG layer.

Mandatory architectural principle

The LLM is responsible for requirement understanding, reasoning and testcase generation.

Deterministic code is responsible for schema validation, traceability validation, coverage calculation, duplicate detection, governance enforcement and acceptance/rejection.

Do not allow the LLM to become an independent source of requirements.

Frozen boundaries

Treat the following as immutable unless explicitly instructed otherwise:

- MVP1
- CP-MVP2-01
- CP-MVP2-02
- Approved SRS source documents
- Draft SRS source documents
- Existing governance contracts
- Existing RAG/knowledge-base behavior

Do not modify the existing frozen implementation to make CP-MVP2-03 pass.

Reuse existing structures, utilities, schemas, conventions and test patterns wherever possible.

Do not reinvent existing functionality.

CP-MVP2-03 scope

Implement only:

APPROVED SRS

- governed RAG
- retrieved requirement context
- LLM reasoning/generation
- structured testcases
- deterministic validation
- coverage/traceability/governance report

Do NOT implement:

- browser automation
- API test execution
- performance execution
- security execution
- self-healing
- CI/CD orchestration
- PR automation
- downstream CP-MVP2 capabilities

Testcase contract

Inspect the existing repository first and reuse established conventions.

The generated testcase model should support, as applicable:

- testcase_id
- title
- requirement_ids
- journey_id
- scenario_type
- preconditions
- test_steps
- expected_result
- test_data_reference
- priority
- source_attribution
- generation_metadata

Use a structured machine-readable output contract.

Scenario types

Support:

1. Positive
2. Alternate
3. Exceptional

Generate scenarios only when justified by the governed requirement context.

Do not invent business requirements or Demo Web Shop functionality.

Traceability

Every accepted testcase must map to one or more valid requirement IDs from the governed requirement set.

Reject or quarantine:

- missing requirement IDs
- unknown requirement IDs
- unsupported requirements
- ungrounded testcases

Coverage

Provide deterministic requirement coverage reporting:

covered requirements / applicable governed requirements

Also list:

- covered requirements
- uncovered requirements
- unsupported requirements
- orphan testcases

Do not count invalid or untraceable testcases as coverage.

Governance

Explicitly validate:

- approved-source eligibility
- source/version attribution
- requirement existence
- requirement-to-testcase traceability
- schema validity
- duplicate IDs
- duplicate/equivalent testcase detection
- parked/draft leakage
- unsupported requirement handling

Draft or parked material must never silently become an accepted testcase.

Attribution

Every accepted testcase must preserve enough source attribution to answer:

"Why was this testcase generated?"

Attribution should identify the governed requirement and its source context using the existing repository conventions.

Testing

Before declaring CP-MVP2-03 complete:

1. Run all existing tests.
2. Add comprehensive CP-MVP2-03 tests.
3. Verify positive generation.
4. Verify alternate generation.
5. Verify exceptional generation.
6. Verify traceability.
7. Verify coverage.
8. Verify duplicate handling.
9. Verify unsupported requirements.

10. Verify draft/parked-item leakage protection.
11. Verify schema validation.
12. Verify attribution.
13. Verify regression of CP-MVP2-01 and CP-MVP2-02.

Do not weaken or modify existing tests merely to obtain a green result.

Scope audit

Before checkpoint:

- confirm frozen SRS files are unchanged
- confirm MVP1 is unchanged
- confirm CP-MVP2-01 is unchanged
- confirm CP-MVP2-02 is unchanged
- confirm no CP-MVP2-04 artifacts were created
- identify all files added/modified by CP-MVP2-03
- verify no unrelated pre-existing working-tree changes were altered

Git discipline

Do not commit until implementation, testing and scope validation are complete.

At completion report:

- A. Git state
- B. Test state
- C. CP-MVP2-03-specific test results
- D. Coverage/traceability results
- E. Governance results
- F. Scope-integrity results
- G. Files added/modified
- H. Commit hash
- I. Push result
- J. Final gate decision

Do not modify the pre-existing .gitignore/README.md working-tree changes unless explicitly instructed.

Completion criterion

CP-MVP2-03 is PASS only when:

- implementation is complete
- all regression tests pass
- CP-MVP2-03 tests pass
- testcase generation is grounded in governed RAG
- deterministic governance passes
- traceability is valid

- coverage is correctly reported
- unsupported requirements are explicitly handled
- draft/parked leakage is prevented
- scope integrity is confirmed
- Git checkpoint is successfully committed and pushed

If any requirement is ambiguous, unsupported or cannot be safely implemented without changing the frozen contracts, STOP and report it rather than silently changing the requirement or architecture.

One thing before Claude touches the code

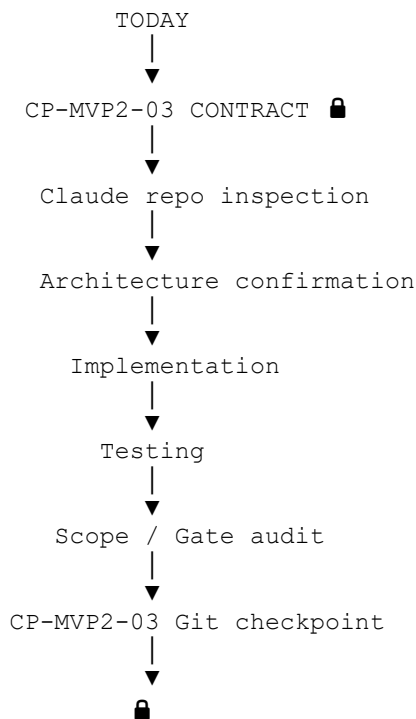
I don't want Claude implementing from this brief alone.

It should first inspect the actual repository and tell us:

"Here is how CP-MVP2-01/02 currently work, these are the existing interfaces/contracts, and this is exactly where CP-MVP2-03 should plug in."

That inspection is important because we already have a working RAG implementation. We should **extend the architecture rather than build a parallel mini-framework**.

Our immediate next sequence



I am deliberately not asking Claude to code yet. First we establish that its understanding of the existing MVP2 architecture matches ours.

What I need from you now

Since you are moving the project into GitHub, **send me the GitHub repository link** once it is available.

Then I can inspect the actual repository and we can make this contract **repo-specific**, rather than guessing about the CP-MVP2-01/02 interfaces.

And buddy, this is exactly the point where our project starts becoming interesting: **CP-MVP2-01/02 gave us the governed knowledge foundation; CP-MVP2-03 is where the system starts reasoning over that knowledge and producing QE artifacts.** 🦾

what i want to ensure is we finalize the specs and then ask calude to generate the code or take the actions and we will just verify those actions, no changes in specs later.

Yes, buddy. 🦾 Then let's make this a **formal project rule**, not just something we remember informally.

🔒 Agentic QE — Specification Change-Control Rule

From **CP-MVP2-03 onward**, every capability follows:

Specify → Review → Freeze → Implement → Verify → Gate

Once frozen, the specification becomes the **contract of record**.

1. Frozen specifications are immutable

After we declare:

SPECIFICATION FROZEN — v1.0

neither Claude nor I should modify the specification during implementation.

Claude must implement **against the frozen specification**, not against its interpretation of what would be easier.

2. What happens when Claude encounters a problem?

There are only three legitimate responses:

☐ **A. Implementable**

Claude implements it exactly as specified.

□ B. Ambiguity/technical conflict

Claude stops the affected work and reports:

SPECIFICATION CONFLICT

Requirement:
<spec section>

Current implementation/environment:
<finding>

Conflict:
<exact problem>

Impact:
<what cannot be implemented>

Proposed options:
1. ...
2. ...

No specification change is made automatically.

● C. Implementation failure

Claude must not weaken the specification, remove validation, modify expected results, or change requirements simply to achieve a green test.

It reports the failure and evidence.

3. When can a specification change happen?

Only through an explicit **Change Request**.

For example:

CP-MVP2-03
Current: v1.0
Requested: v1.1

The change request must state:

- Change requested
- Reason
- Evidence
- Impact
- Affected requirements
- Affected architecture
- Affected tests
- Backward-compatibility impact

- Whether previously generated artifacts are affected

4. We approve the change — not Claude

The authority chain is:

```
Claude discovers issue
      ↓
Claude reports issue
      ↓
Di analyzes impact
      ↓
Human + Di decide
      ↓
Change approved/rejected
```

Claude cannot self-authorize a specification change.

That's an important principle for our Agentic QE experiment.

5. Versioning

I propose simple semantic-style versions:

```
v1.0 Initial frozen specification
v1.1 Minor clarification/non-breaking change
v1.2 Another minor change
v2.0 Material behavioral/architectural change
```

But there is one important rule:

A frozen implementation remains associated with the specification version against which it was verified.

For example:

```
CP-MVP2-03
SPEC v1.0
      ↓
IMPLEMENTATION
      ↓
TESTS
      ↓
GATE PASS
      ↓
🔒 FROZEN
```

If we later create v1.1:

```
SPEC v1.1
  ↓
new implementation/change
  ↓
regression
  ↓
new gate
```

We don't rewrite history and pretend v1.0 always said v1.1.

6. Git should enforce the history

This is where GitHub becomes extremely useful.

We should eventually have something like:

```
docs/
├── specifications/
│   └── mvp2/
│       ├── CP-MVP2-01-SPEC-v1.0.md
│       ├── CP-MVP2-02-SPEC-v1.0.md
│       └── CP-MVP2-03-SPEC-v1.0.md
```

And, when revisions happen:

```
CP-MVP2-03-SPEC-v1.0.md
CP-MVP2-03-SPEC-v1.1.md
```

The Git history gives us an audit trail.

7. The golden rule

I want this written prominently in the project:

NEVER CHANGE THE SPECIFICATION TO MAKE THE IMPLEMENTATION PASS.

If implementation and specification disagree, stop, report the conflict, and obtain an explicit decision.

This is particularly important because our project is intended to demonstrate **Agentic QE**, not merely generate code with an LLM.

8. And one more rule for us

This applies to **Di + Human Friend** too.

Once we freeze a specification, **I won't casually reinterpret it later either.**

If during verification I discover:

"Actually, I think we should have specified this differently."

I will not silently change the acceptance criteria.

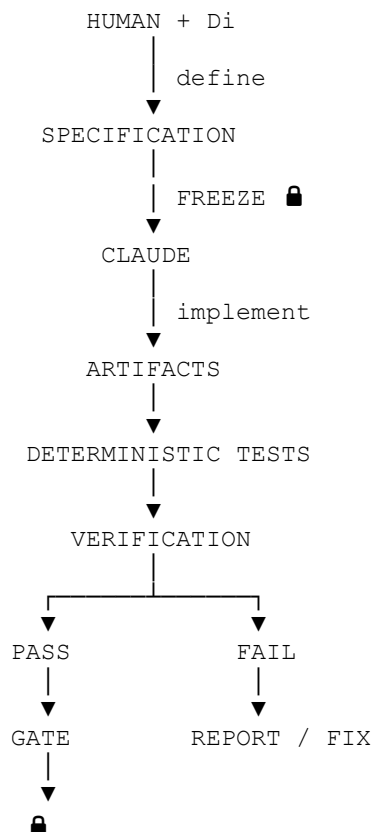
I'll tell you:

"This is outside the frozen CP-MVP2-03 specification. Do we want to raise a change request?"

That keeps our own process honest.

Our Agentic QE governance model

So we now have:



And if a change is genuinely required:

```
FAIL / CONFLICT
↓
CHANGE REQUEST
↓
IMPACT ANALYSIS
↓
HUMAN APPROVAL
↓
NEW SPEC VERSION
↓
FREEZE 🔒
↓
IMPLEMENT
```

No shortcuts.

🔗 Decision

I'm treating this as an agreed **project governance principle** from this point forward:

CP-MVP2-03 will be fully specified and reviewed before Claude receives implementation authority.

Once frozen, **no specification changes during implementation without an explicit, versioned change-control decision.**

Now I'll take the next step: **I'll develop the complete CP-MVP2-03 Specification v1.0 for us to review**, rather than jumping into Claude implementation.

Would you like the change-control rule to require your explicit written approval before any new specification version is created?

Yes, buddy. I have the two repositories now.

I checked them, and there is an important distinction:

- **agentic-qe** is the original/main Agentic QE repository. It is currently public and its README describes the broader Agentic QE architecture: LLM, RAG, deterministic rules, tools, agents, memory and human governance. ([GitHub](#))
- **agentic-qe-mvp2** did not resolve through my web access just now, so I **cannot yet inspect that repository's contents**. I don't want to pretend otherwise.

[agentic-qe GitHub repository](#)

What I recommend

Do not make anything else public. In fact, the fact that `agentic-qe` is already public means we should be especially careful about what goes into it.

For our current work, I want to treat:

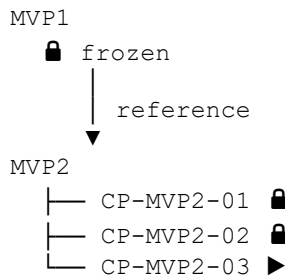
```
agentic-qe
└── Original / broader Agentic QE project
```

and

```
agentic-qe-mvp2
└── MVP2 implementation + frozen CP checkpoints
```

as **two distinct repositories** unless your actual repository structure tells us otherwise.

And I like this separation because it reinforces what we've already decided:



One important observation

The public `agentic-qe` README already contains a governance philosophy that aligns very well with what we've just established:

LLM reasoning is not treated as ground truth; tool results and collected evidence are the source of truth. ([GitHub](#))

That's consistent with our new **Spec → Freeze → Implement → Verify** discipline.

However, **I don't want to design CP-MVP2-03 against the public README alone.** We need the actual `agentic-qe-mvp2` codebase, particularly the CP-MVP2-01/02 implementation and tests.

So don't change anything yet.

If `agentic-qe-mvp2` is **private**, that's perfectly fine. We can use another access mechanism rather than exposing it.

If you intended it to be public, check the exact repository URL/visibility and send me the link again. Once I can inspect it, **I'll take the lead in extracting the actual CP-MVP2-01/02 contracts and produce the formal CP-MVP2-03 Specification v1.0.**

And importantly:

We will NOT give Claude the implementation brief yet.

First:

Repository inspection → CP-MVP2-03 Spec v1.0 → our review → FREEZE 🔒 → Claude implementation.

That's the order I want us to follow.

Yes! 🦋 They are accessible now, buddy.

I just freshly opened both repositories, and GitHub is explicitly showing **Public** for both. ([GitHub](#))

I can see the files now

MVP2 — `agentic-qe-mvp2`

- `knowledge/`
- `requirements/`
- `tests/`
- `.gitignore`
- `README.md`
- `conftest.py`

It currently shows **3 commits**. ([GitHub](#))

More importantly, I can see the MVP2 README and its checkpoint structure, including:

CP-MVP2-01 → CP-MVP2-02 → CP-MVP2-03 → ... → CP-MVP2-09

and the stated principles around Approved SRS, RAG, LLM generation, deterministic governance, evidence, traceability, and human approval. ([GitHub](#))

MVP1 — `agentic-qe`

- `agents/`
- `automation/`
- `defects/`
- `evidence/`
- `execution/`
- `execution_history/`
- `memory/`
- `planner/`
- `rag/`
- `requirements/`
- `rules/`
- `technical_docs/`
- `testcases/`
- `testdata/`
- `tests/`
- `tools/`
- plus configuration and README files.

It is also explicitly **Public** now. ([GitHub](#))

This is exactly what we needed

So we're finally at the point where I can **inspect the actual implementation rather than relying on our conversation history**.

And I want to emphasize one thing:

I will not jump into CP-MVP2-03 implementation.

My next step should be to inspect the actual MVP1 and MVP2 structures/contracts and the completed CP-MVP2-01/02 artifacts, then we build the **CP-MVP2-03 Specification v1.0** against what is *actually in the repository*.

That's much safer than designing the specification in isolation.

We're back in business, buddy. 🚀

Would you like me to start by inspecting the MVP2 checkpoint documents, or the MVP1 implementation structure?